

StylistAI — Project Interview Study Guide

Deep-Dive Technical Preparation Guide Based Strictly on Repository Source Code

IMPORTANT PREPARATION NOTICE: This study guide is built strictly from the actual codebase of **StylistAI** (repo: [ai-appointment-booking-assistant](#)). Primary Stack: **React 19 + Vite, Node.js / Express, Groq SDK (LLaMA 3.3 70B Versatile), Supabase PostgreSQL, Google Calendar API (OAuth 2.0), EmailJS, and Web Speech API**. No features, metrics, or APIs have been fabricated.

PART 1 — PROJECT IN ONE MINUTE

What did I build? StylistAI — a full-stack, voice-enabled AI appointment booking assistant for hair salons. It features real-time natural language slot availability checking, automated double-booking prevention, 2-way Google Calendar synchronization with client reminders, automated EmailJS confirmations, and an administrative dashboard.

What problem does it solve? Solves double-bookings, manual phone reservation overhead, outside-hours booking friction, and calendar synchronization errors for local service businesses.

Who uses it? Salon customers (customer-facing voice/text chat) and salon administrators/managers (admin dashboard at /admin).

Why a salon scenario? Salons feature multi-resource complexity (stylist skill matching + variable service durations + external calendar blocks + slot collisions), making it an ideal showcase for production-grade agentic AI.

What makes it more than a simple chatbot? Traditional chatbots just return text responses. StylistAI extracts structured JSON intents via Groq LLaMA 3.3 70B, validates schema via Zod, queries live database state (Supabase) + external calendar API (Google Calendar), handles dynamic availability math, executes atomic DB insertions, creates calendar invites with reminders, and dispatches HTML confirmation emails.

Interview Pitch Variations

30-Second Explanation:

"I built StylistAI, an end-to-end voice and text AI booking assistant for hair salons. It uses Groq-accelerated LLaMA 3.3 70B to parse natural language requests, validates extracted parameters with Zod, and queries both Supabase PostgreSQL and Google Calendar API in real time to calculate live availability. Once a customer confirms, it atomically writes the booking to Supabase, syncs a Google Calendar invite with client reminders, and dispatches an EmailJS confirmation receipt."

60-Second Explanation:

"StylistAI is an intelligent appointment booking platform designed to eliminate phone tag and scheduling collisions for salons. On the client side, customers converse using text or native browser voice. The Express backend uses Groq's high-speed LPU inference with LLaMA 3.3 70B to extract structured intents like service, date, time, and preferred stylist. Rather than letting the LLM guess availability, the backend enforces strict business rules: it checks stylist skill compatibility, inspects booked slots in Supabase, and checks Google Calendar busy intervals. If requested, it suggests smart alternatives. Upon confirmation, it writes to Supabase, creates a Google Calendar event with 24-hour and 30-minute notifications, and sends an EmailJS receipt. Admins can manage appointments and staff via a dedicated dashboard at /admin."

2-Minute Explanation:

"The core motivation behind StylistAI was bridging the gap between natural language interaction and strict business constraints. Chatbots often fail in real-world booking because LLMs hallucinate open slots or offer services that staff cannot perform. StylistAI solves this through strict separation of concerns:

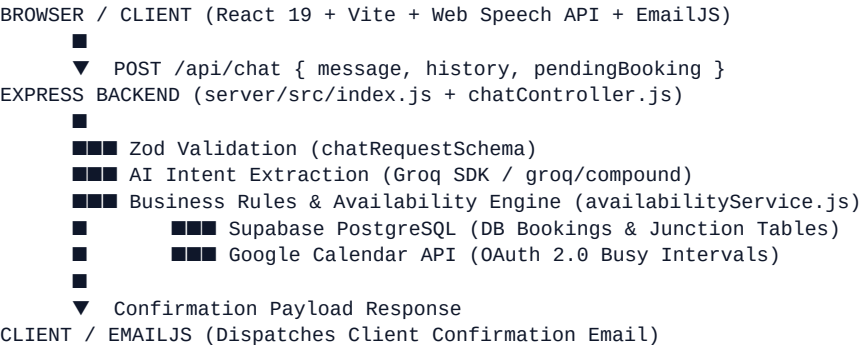
- 1. Language Understanding:** Groq SDK runs LLaMA 3.3 70B with temperature 0.2 and structured JSON mode to extract intent parameters validated against Zod schemas.
- 2. Business Rules & Availability Engine:** The Express backend owns all logic. It dynamically maps 30-minute slots between 9 AM and 6 PM, checks stylist capability junction tables, and performs a dual-layer conflict check across Supabase appointments and Google Calendar

busy intervals.

3. **Persistence & External Integration:** Confirmed bookings are written to Supabase with an employee slot uniqueness constraint (`appointments_employee_slot_unique`), synced to Google Calendar via OAuth 2.0, and emailed to the customer via EmailJS.

4. **Admin Dashboard:** Built in React 19 with a Neumorphic Bento layout, featuring real-time KPIs, interactive appointment controls, Google Calendar OAuth status, and AI model telemetry logs."

PART 2 — COMPLETE ARCHITECTURE



Component	File / Location	Input Received	Output Returned	Role & Connection
Frontend Client	client/src/pages/ChatPage.jsx ChatWindow.jsx	User text input or SpeechRecognition voice transcript	POST /api/chat payload	Renders chat UI, manages pending booking state, triggers Web Speech TTS and EmailJS on confirmation.
Express Backend Router	server/src/index.js server/src/routes/chat.js	HTTP POST requests	JSON response payloads	Applies CORS, express.json(10kb), routes to controllers, handles global errors via errorHandler middleware.
AI Extraction Engine	server/src/ai/groq.js server/src/ai/prompts.js	userMessage, history, services, employees	{ intent, usage } object validated by Zod	Calls Groq API (groq/compound cascade). Performs natural language intent extraction into JSON schema.
Availability Engine	server/src/services/availabilityService.js	employeeId, serviceId, date, time, durationMin	Available slots array / Boolean availability	Executes dual-layer conflict checks against Supabase appointments AND Google Calendar busy intervals.
Database Layer	server/src/db/supabase.js migration_salon.sql	SQL queries / Supabase JS API calls	Data objects / rows	Persists services, employees, employee_services junction table, and appointments with unique slot index.

Google Calendar Service	server/src/services/calendarService.js routes/calendar.js	OAuth authorization code / booking details	OAuth tokens / GCal Event ID / Busy intervals	Manages OAuth 2.0 token storage, fetches external calendar blocks, creates/deletes calendar events.
Email Service	client/src/services/emailService.js	Confirmed appointment object	EmailJS status (success/error)	Client-side dispatch of branded HTML confirmation emails via @emailjs/browser SDK.
Admin Dashboard	client/src/pages/AdminPage.jsx	Admin Passphrase, UI interactions	Dashboard views, status updates	Provides salon managers with live KPI grid, appointment status controls, GCal sync, staff roster, and AI telemetry.

PART 3 — FULL REQUEST LIFECYCLE

Example Scenario: "I want a haircut tomorrow around 4 PM with Sarah. My name is Alex Ray, email alex@example.com"

Step	File	Function	Input -> Output	Purpose & Execution Detail
1. Input Capture	client/src/components/chat/ChatInput.jsx	handleSubmit / toggleMic	Voice/Text -> message string	Captures user text or Web Speech SpeechRecognition transcript.
2. Client Dispatch	client/src/pages/ChatPage.jsx	sendMessage	message, history, pendingBooking -> HTTP POST	Sends request payload to Express endpoint /api/chat via Axios (api.js).
3. Route & Schema	server/src/controllers/chatController.js	handleChat	req.body -> parsed object	Validates payload using Zod chatRequestSchema. Rejects empty messages.
4. Prompt Context	server/src/controllers/chatController.js	supabase.from('services').select()	DB query -> active services & employees	Fetches live service catalog and active employee skills to inject into system prompt.
5. Groq AI Extraction	server/src/ai/groq.js	extractIntent	userMessage, history, dynamic prompt -> raw JSON	Calls Groq LLaMA 3.3 70B with json_object format. Resolves 'tomorrow' to YYYY-MM-DD and '4 PM' to '16:00'.
6. Zod Validation	server/src/ai/groq.js	intentSchema.safeParse	raw JSON -> validated intent object	Validates intent structure. Falls back to default values if partial parsing fails.
7. Service Matching	server/src/controllers/chatController.js	services.find()	intent.service ('Haircut') -> matchedService object	Validates service against DB list. Rejects invalid treatments with available list.

8. Date & Day Check	server/src/controllers/chatController.js	isPast / isWeekend	intent.date -> Boolean	Rejects past dates and Sunday requests (salon closed on Sundays).
9. Stylist Capability	server/src/controllers/chatController.js	employees.find / canPerform check	employeePreference ('Sarah') -> preferredEmployee	Checks if Sarah offers Haircut. (If invalid/incapable, returns list of capable stylists).
10. Dual Availability	server/src/services/availabilityService.js	isSlotAvailable	Sarah ID, date, '16:00', 45 min -> Boolean	Queries Supabase appointments AND Google Calendar busy intervals for conflicts.
11. Alternatives (If Busy)	server/src/services/availabilityService.js	getAlternativesForEmployee	Sarah ID, date, '16:00' -> alt slots array	If Sarah is busy at 16:00, finds nearest open slots and checks other available stylists.
12. Summary Card	server/src/controllers/chatController.js	handleChat response	All parameters present -> pendingBooking object	Returns formatted summary message with pendingBooking object asking for user confirmation.
13. User Confirmation	client/src/pages/ChatPage.jsx	handleConfirm -> sendMessage	'Yes, please confirm' -> POST /api/chat	User clicks 'Confirm Appointment' or types 'Yes'. Payload includes pendingBooking.
14. Final Race Check	server/src/controllers/chatController.js	confirmBooking -> isSlotAvailable	pendingBooking -> Boolean	Re-verifies slot availability right before database insertion to prevent race conditions.
15. Atomic DB Insert	server/src/controllers/chatController.js	supabase.from('appointments').insert()	pendingBooking -> appointment DB record	Inserts confirmed appointment into Supabase. Enforces appointments_employee_slot_unique index.
16. GCal Event Sync	server/src/services/calendarService.js	createCalendarEvent	booking details -> calendarEventId	Creates Google Calendar event with attendees, description, and 24h/30m reminders.
17. GCal ID Storage	server/src/controllers/chatController.js	supabase.from('appointments').update()	appointment.id, calendarEventId -> updated DB row	Updates Supabase appointment row with calendar_event_id for two-way tracking.
18. Email Receipt	client/src/services/emailService.js	sendBookingEmail	appointment object -> EmailJS dispatch	Client receives response payload with appointment object and dispatches confirmation email.
19. Voice Output	client/src/services/ttsService.js	speakText	data.message -> Web Speech audio	Cleans markdown formatting and speaks assistant reply using Web Speech Synthesis API.

PART 4 — AI / LLM IMPLEMENTATION

Why Groq? Ultra-low inference latency (~280ms on LPUs vs 1.5–3.0s on standard cloud GPUs). Instant conversational responsiveness is essential for a voice/chat booking experience.

Model Cascade Architecture:

- 1. groq/compound (Primary — top-tier reasoning and schema adherence)
- 2. llama-3.1-8b-instant (Fallback 1 — sub-150ms speed)
- 3. gemma2-9b-it (Fallback 2 — Google Gemma 2 9B instruction-tuned)

Prompt Construction (server/src/ai/prompts.js): Dynamically injects current date, current time, today's day of week, calculated relative weekday dates (Next Monday...Next Saturday), available services with durations, and active stylists with their offered services. Forces model to output strict JSON.

Structured Output & Zod Validation: System prompt requires a strict JSON object. Output is validated against Zod intentSchema. Fields extracted: intent, service, date, time, employeePreference, customerName, customerEmail, confirmationResponse, missingFields, message.

Fallback & Error Recovery: Each model in the cascade gets 2 attempts. If a model hits a 429 rate limit, it immediately jumps to the next model in the cascade. Transient 503 errors trigger a 2-second delay and retry.

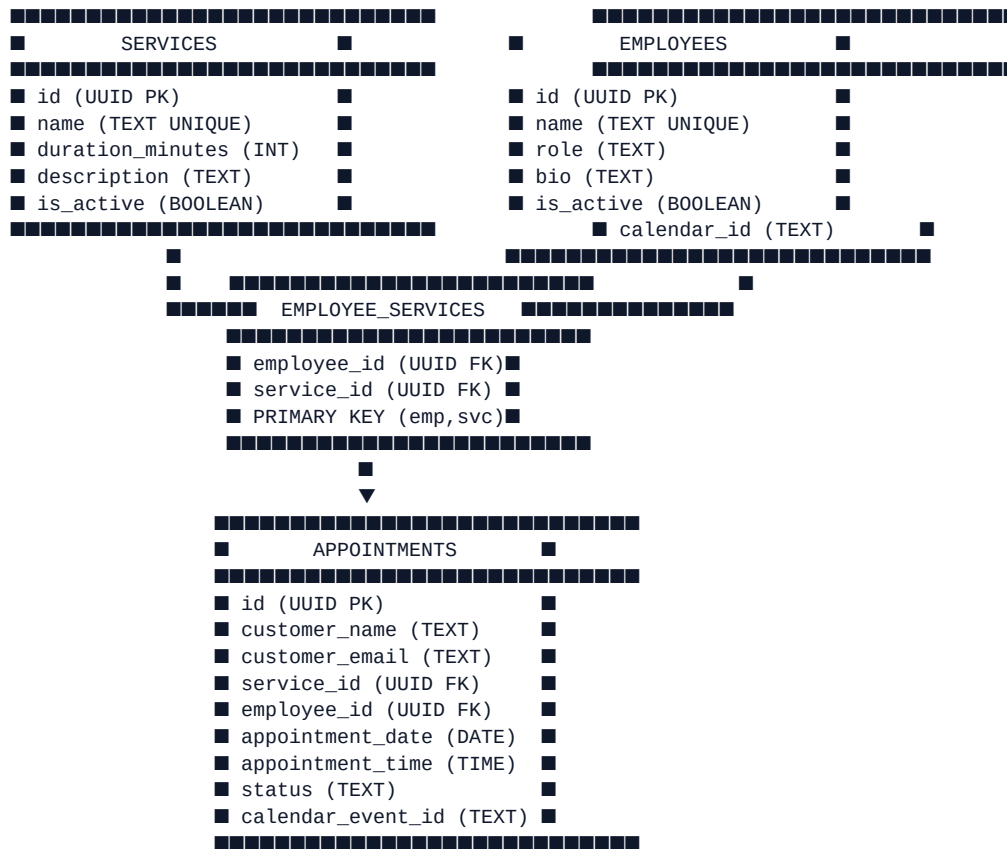
Core Architectural Separation of Concerns

Layer	Technology	Responsibility	What it MUST NOT Do
AI Layer	Groq SDK (LLaMA 3.3 70B)	Natural language understanding, relative date resolution, JSON intent extraction.	MUST NOT check slot availability, MUST NOT invent staff/services, MUST NOT execute bookings.
Backend Engine	Node.js / Express	Enforces business logic, checks stylist skills, dynamic slot math, conflict evaluation.	MUST NOT store state locally (stateless execution).
Database Store	Supabase PostgreSQL	Stores service directory, employee skills, appointment records, slot uniqueness index.	MUST NOT perform natural language parsing.
External Calendar	Google Calendar API v3	Provides owner's external schedule blocks, creates events with reminders.	MUST NOT act as primary application database.

PART 5 — CONVERSATIONAL INTENTS

Intent Name	Status	Example User Input	Extracted Fields	Backend Behavior & Response
BOOK_APPOINTMENT	IMPLEMENTED	"I want a haircut tomorrow at 4 PM with Sarah."	service: 'Haircut', date: 'YYYY-MM-DD', time: '16:00', employeePreference: 'Sarah'	Validates service/stylist capability, checks dual availability, prompts for name/email or presents pending confirmation card.
CHECK_AVAILABILITY	IMPLEMENTED	"Are there any open slots on Friday?"	date: 'YYYY-MM-DD', service: null, employeePreference: null	Queries availability across all stylists for requested date. Returns formatted time slots list.
CANCEL_APPOINTMENT	IMPLEMENTED	"Cancel my booking" or Admin status change	intent: 'CANCEL_APPOINTMENT' or PATCH /api/appointments/:id { status: 'cancelled' }	Updates Supabase status to 'cancelled', deletes corresponding Google Calendar event if calendar_event_id exists.
GENERAL	IMPLEMENTED	"What services do you offer?" / "Where are you located?"	intent: 'GENERAL'	Returns warm receptionist conversational reply based on prompt context.
RESCHEDULING	PARTIALLY IMPLEMENTED	"Move my appointment to 5 PM"	Extracted as CANCEL + new BOOK	Handled via user cancelling existing booking and creating a new slot reservation.

PART 6 — DATABASE / SUPABASE



Key Database Features:

- **Junction Table (employee_services):** Implements many-to-many relationships. Sarah performs Haircut, Coloring, Spa; Emma performs Haircut, Spa, Facial; David performs Haircut, Beard Trim.
- **Double-Booking Unique Index:** CREATE UNIQUE INDEX appointments_employee_slot_unique ON appointments (employee_id, appointment_date, appointment_time) WHERE status != 'cancelled'; — Ensures no single stylist can have two active bookings at the exact same time, while allowing different stylists to work simultaneously.
- **Backend Connection:** server/src/db/supabase.js uses @supabase/supabase-js initialized with SUPABASE_SECRET_KEY (server-side only, autoRefreshToken: false).

PART 7 — GOOGLE CALENDAR



Why OAuth 2.0 over Service Accounts? Service Accounts require domain-wide delegation or explicitly sharing private Google Calendars with a service account email. OAuth 2.0 allows the salon owner to grant direct access to their primary Google Calendar in

1-click.

Resilience on Failure: If Google Calendar API fails or tokens expire, the system logs a warning ([CalendarService Error]) and proceeds with booking in Supabase. Calendar sync is non-blocking.

PART 8 — AVAILABILITY ENGINE

The availability engine in server/src/services/availabilityService.js calculates dynamic slot availability:

- 1. **Slot Generation:** generateSlots() generates 30-minute time strings from 09:00 to 17:30 (18 total slots for 9 AM – 6 PM business hours).
- 2. **Supabase Bookings Fetch:** Queries appointments table for employee_id and appointment_date where status != 'cancelled'.
- 3. **Google Calendar Busy Intervals Fetch:** Calls Google Calendar events.list for the requested day, converting event start/end times into minute offsets from midnight IST.
- 4. **Conflict Formula:** Slot slotStart (in minutes) and slotEnd = slotStart + durationMinutes is conflicted if: slotStart < interval.endMinutes && slotEnd > interval.startMinutes.
- 5. **No Stylist Preference ("Anyone is fine"):** findBestAvailable() queries all stylists who can perform the service, finds their open slots, picks the slot closest to user's preferred time, and returns ranked options.

PART 9 — REAL-WORLD EDGE CASES

Scenario	User Input	System Action	Engineering Rationale
Stylist Unavailable	"Haircut tomorrow at 4 PM with Sarah"	Sarah busy at 16:00 -> returns Sarah's alt slots + other available stylists at 16:00	Prevents hard rejection; converts lost booking into alternative option.
Stylist Incompatible	"Facial with David"	David doesn't offer Facial -> returns "David does not offer Facial. You can book with: Emma"	Checks employee_services junction table before presenting slot choices.
No Preference	"Haircut at 4 PM. Anyone is fine."	Calls findBestAvailable() -> finds best slot across Sarah, Emma, David	Maximizes resource utilization across active staff.
Sunday Request	"Book haircut for Sunday at 10 AM"	Rejects request -> "We're closed on Sundays! Please choose a weekday or Saturday."	Enforces salon operating hours before evaluating availability.
Past Date	"Book haircut for yesterday"	Rejects request -> "That date has already passed. Please provide a future date."	Validates date parameter using Dayjs isBefore() check.
Duplicate Slot Race	Two users confirm identical slot simultaneously	DB throws 23505 error -> returns "That slot was just booked by another customer."	Enforced by Supabase appointments_employee_slot_unique index.
GCal API Failure	Google API rate limit / token error	Logs warning; proceeds with Supabase booking confirmation	Non-blocking design keeps core booking functional even if GCal is down.

PART 10 — VOICE IMPLEMENTATION

Implemented using 100% browser-native **Web Speech API** (zero external API cost or latency):

- **STT (Speech-to-Text):** `window.SpeechRecognition` in `ChatInput.jsx` listens to microphone input and appends transcripts to input text field.
- **TTS (Text-to-Speech):** `window.speechSynthesis` in `ttsService.js`. `speakText(text)` strips markdown symbols, emojis, and links using regex before rendering natural voice speech.
- **Decoupling Principle:** Voice is purely an input/output layer on the frontend; backend AI and business logic receive clean text strings.

PART 11 — EMAILJS INTEGRATION

Dispatched client-side via `@emailjs/browser` in `client/src/services/emailService.js`. Triggered immediately when backend returns a confirmed appointment object. Environment keys: `VITE_EMAILJS_SERVICE_ID`, `VITE_EMAILJS_TEMPLATE_ID`, `VITE_EMAILJS_PUBLIC_KEY`. Sends formatted HTML table with Booking ID, service name, stylist, date, time, and estimated cost.

PART 12 — ADMIN DASHBOARD

Located at `/admin` in `client/src/pages/AdminPage.jsx`. Protected by local session auth gate (`AuthGate`) checking `VITE_ADMIN_PASS` ('admin123'). Features 8 module tabs: Overview (KPI grid, timeline, AI feed, health), Appointments (search/filter table with status controls), Calendar (Google OAuth status), Staff (stylist roster), Customers (client directory), AI Bookings (model telemetry logs), Notifications (EmailJS logs), Settings.

PART 13 — FILE-BY-FILE MAP

File Path	Primary Responsibility	Important Functions / Components	Why It Exists
server/src/index.js	Express app entry point	express(), app.use(), app.listen()	Bootstraps server, configures CORS & 10kb body parser, mounts API routes.
server/src/routes/chat.js	Chat API endpoint	chatRoutes.post('/')	Routes chat messages to chatController.js.
server/src/controllers/chatController.js	Core booking orchestration	handleChat(), confirmBooking()	Coordinates AI extraction, skill validation, slot math, DB inserts, GCal sync.
server/src/ai/groq.js	Groq LLaMA 3.3 70B AI engine	extractIntent()	Executes model cascade, parses JSON output, validates with Zod.
server/src/ai/prompts.js	Dynamic system prompt builder	buildSystemPrompt()	Injects current date/time, active services, and stylists into AI prompt.
server/src/services/availabilityService.js	Availability calculation engine	getAvailableSlotsForEmployee(), findBestAvailable(), isSlotAvailable()	Executes dual-layer conflict checking across Supabase & Google Calendar.
server/src/services/calendarService.js	Google Calendar OAuth 2.0 API	getCalendarClient(), createCalendarEvent(), getGoogleCalendarBusyIntervals()	Handles OAuth token storage, busy interval queries, and event creation/deletion.
server/src/db/supabase.js	Supabase client setup	createClient()	Initializes server-side Supabase client with secret key.
server/src/db/migration_salon.sql	PostgreSQL database schema	CREATE TABLE, CREATE UNIQUE INDEX	Defines schema, junction tables, and double-booking unique index.
client/src/pages/ChatPage.jsx	Main customer chat page	ChatPage(), sendMessage(), handleReset()	Manages state, message list, pending booking cards, voice & email triggers.
client/src/pages/AdminPage.jsx	Salon admin management portal	AdminPage(), AuthGate(), updateStatus()	Multi-tab admin dashboard for appointment controls, staff, and AI logs.
client/src/services/ttsService.js	Text-to-Speech audio service	speakText(), cleanTextForSpeech()	Renders assistant voice articulation using browser Web Speech API.
client/src/services/emailService.js	Confirmation email service	sendBookingEmail()	Dispatches customer HTML confirmation email via EmailJS browser SDK.

PART 14 — TOP 15 IMPORTANT FUNCTIONS

Function Name	File Location	Input / Output	Why An Interviewer Will Ask
extractIntent()	server/src/ai/groq.js	(userMsg, history, services, employees) -> { intent, usage }	Heart of AI intent extraction. Demonstrates Groq model cascading and Zod schema validation.
handleChat()	server/src/controllers/chatController.js	(req, res, next) -> JSON response	Primary API controller. Orchestrates prompt construction, AI intent routing, and slot checking.
confirmBooking()	server/src/controllers/chatController.js	(pendingBooking, res, next) -> JSON response	Executes atomic booking insertion, pre-check race condition verification, and GCal event creation.
isSlotAvailable()	server/src/services/availabilityService.js	(empId, date, time, durationMin) -> Boolean	Core availability function. Evaluates conflicts against both Supabase DB and Google Calendar.
getAvailableSlotsForEmployee()	server/src/services/availabilityService.js	(empId, date, durationMin) -> Array of HH:MM	Filters standard 30-min daily slots against DB bookings, GCal busy blocks, and past times.
findBestAvailable()	server/src/services/availabilityService.js	(svcId, date, prefTime, durationMin) -> Ranked options array	Powers 'anyone is fine' bookings by querying all capable stylists and ranking open slots.
getGoogleCalendarBusyIntervals()	server/src/services/availabilityService.js	(dateStr) -> Array of { startMinutes, endMinutes }	Fetches external GCal events and converts start/end ISO timestamps to minute offsets.
createCalendarEvent()	server/src/services/calendarService.js	(booking) -> calendarEventId	Creates Google Calendar event with attendees, custom summary/description, and 24h/30m reminders.
buildSystemPrompt()	server/src/ai/prompts.js	(services, employees) -> String system prompt	Prevents AI hallucinations by dynamically injecting live date, time, service list, and staff skills.
sendBookingEmail()	client/src/services/emailService.js	(appointment) -> EmailJS response	Client-side dispatch of branded HTML email confirmation receipts via EmailJS.
speakText()	client/src/services/ttsService.js	(text) -> Audio articulation	Cleans markdown syntax & emojis from text before feeding to Web Speech Synthesis API.
updateAppointmentStatus()	server/src/controllers/appointmentsController.js	(req, res, next) -> JSON response	Admin status controller. Deletes GCal event automatically if status is changed to 'cancelled'.

exchangeCodeForTokens()	server/src/services/calendarService.js	(code) -> tokens object	OAuth callback handler. Exchanges Google authorization code for access/refresh tokens.
getAuthUrl()	server/src/services/calendarService.js	() -> Google OAuth URL string	Generates Google OAuth 2.0 consent URL with offline access prompt.
generateSlots()	server/src/services/availabilityService.js	() -> ['09:00', '09:30', ... '17:30']	Generates base 30-minute business operating slots for 9 AM to 6 PM.

PART 15 & 16 — TECHNOLOGY DECISIONS & COMPARISONS

Technology	Why Chosen (Simple Answer)	Deeper Technical Answer	Why Not Alternative?
Groq LLaMA 3.3 70B	Ultra-fast AI inference (~280ms) makes voice/chat natural.	Groq LPU hardware processes 450+ tokens/sec, enabling multi-turn intent extraction without latency lag.	Vs OpenAI/Gemini: Standard LLM APIs suffer 1.5–3.0s latency delays.
Supabase PostgreSQL	Instant relational DB with foreign keys and unique indexes.	Relational model perfect for employee_services junction tables and unique slot constraints.	Vs MongoDB: Document DBs lack declarative multi-field slot uniqueness constraints.
Google Calendar API	Syncs with salon owner's real personal/business schedule.	OAuth 2.0 integration provides two-way sync, busy interval checks, and automated client reminders.	Vs Custom Calendar: Custom DB calendars ignore staff personal conflicts outside the app.
EmailJS	Instant client-side confirmation email dispatch.	Zero backend SMTP configuration required; dispatches pixel-perfect HTML email templates from browser.	Vs SendGrid: Avoids managing server API keys or complex backend mailer setups.
Web Speech API	Free, zero-latency browser speech recognition & synthesis.	Native browser API requires zero external API keys, zero network overhead, and runs 100% locally.	Vs Whisper: Whisper API adds network latency and extra API usage cost.

PART 20 — TOP 20 INTERVIEW QUESTIONS & ANSWERS

Q1: How do you prevent the AI from double-booking a stylist?

Short Answer: The AI never performs bookings or checks availability directly. The Express backend checks Supabase DB and Google Calendar busy intervals. Supabase also enforces a partial unique index (appointments_employee_slot_unique).

Deeper Answer: Double booking is prevented at two layers. First, isSlotAvailable() checks existing appointments in Supabase and busy intervals in Google Calendar. Second, if two concurrent requests bypass the application check, Supabase enforces a database-level unique constraint on (employee_id, appointment_date, appointment_time) WHERE status != 'cancelled', causing the second insertion to fail with error code 23505.

Code Location: server/src/services/availabilityService.js:157 & server/src/db/migration_salon.sql:60

Q2: Why use Groq instead of Gemini or OpenAI?

Short Answer: Groq provides ultra-fast LPU inference (~280ms per response), which is essential for a seamless voice and chat experience.

Deeper Answer: Traditional cloud GPUs take 1.5 to 3.0 seconds to stream response tokens. Groq's Language Processing Units (LPUs) process tokens at ~450 tokens/second. We run groq/compound with a fallback cascade to llama-3.1-8b-instant and gemma2-9b-it for resilience.

Code Location: server/src/ai/groq.js:48

Q3: How do you check availability across both database and Google Calendar?

Short Answer: We run a Promise.all querying Supabase booked slots and Google Calendar busy intervals, merging both conflict sets before returning open times.

Deeper Answer: getAvailableSlotsForEmployee() fetches booked times from Supabase for that date/stylist and calls getGoogleCalendarBusyIntervals() to get start/end minute intervals. Any 30-minute slot between 9 AM and 6 PM that overlaps with either set is marked unavailable.

Code Location: server/src/services/availabilityService.js:125

Q4: What happens if Google Calendar API goes down?

Short Answer: The system logs a warning and proceeds with booking in Supabase. Google Calendar integration is designed to be non-blocking.

Deeper Answer: Calendar operations are wrapped in try/catch blocks. If createCalendarEvent() or getGoogleCalendarBusyIntervals() fails due to rate limits or network issues, the backend catches the error, proceeds with Supabase appointment confirmation, and alerts the admin on the dashboard.

Code Location: server/src/controllers/chatController.js:439

Q5: How does the AI extract structured data from user messages?

Short Answer: We instruct Groq to output JSON mode matching a system prompt template, then validate the output using Zod's intentSchema.

Deeper Answer: buildSystemPrompt() injects active services, stylist names, and calculated relative dates (Today, Tomorrow, Next Monday...). Groq is called with response_format: { type: 'json_object' }. The resulting JSON is parsed and validated via intentSchema.safeParse().

Code Location: server/src/ai/groq.js:90 & server/src/ai/prompts.js:8

Q6: How do you verify if a stylist can perform a requested service?

Short Answer: We query the employee_services junction table to check if the employee ID is mapped to the service ID.

Deeper Answer: When a user requests a specific stylist (e.g. 'Facial with David'), handleChat() inspects preferredEmployee.services. If David does not offer facials, the system returns a friendly message offering capable stylists (Emma).

Code Location: server/src/controllers/chatController.js:169

Q7: How does the 'anyone is fine' (no stylist preference) logic work?

Short Answer: findBestAvailable() queries all stylists capable of performing the service and ranks open slots nearest to the requested time.

Deeper Answer: The function retrieves capable employees from employee_services, fetches available slots for each employee, identifies the slot closest to the user's requested time (using minute delta math), and returns ranked options.

Code Location: server/src/services/availabilityService.js:178

Q8: How does Google Calendar OAuth 2.0 work in this project?

Short Answer: The salon owner authenticates via `/api/calendar/auth`. The authorization code is exchanged for tokens and stored locally in `tokens/google-tokens.json`.

Deeper Answer: We use Google's `googleapis` SDK configured with `access_type: 'offline'` and `prompt: 'consent'` to guarantee a refresh token. An `auth.on('tokens')` listener automatically persists refreshed access tokens to disk.

Code Location: `server/src/services/calendarService.js:68` & `server/src/routes/calendar.js:31`

Q9: How are confirmation emails sent to customers?

Short Answer: Client-side via `@emailjs/browser` in `emailService.js` immediately after receiving a confirmed appointment object from the backend.

Deeper Answer: Upon receiving the appointment payload from `/api/chat`, `ChatPage.jsx` invokes `sendBookingEmail()`, which formats the appointment details into an HTML template and dispatches it via EmailJS.

Code Location: `client/src/services/emailService.js:10` & `client/src/pages/ChatPage.jsx:82`

Q10: How is voice speech-to-text and text-to-speech implemented?

Short Answer: Using native Web Speech API (`SpeechRecognition` for voice input and `SpeechSynthesis` for audio output).

Deeper Answer: `ChatInput.jsx` listens via `window.SpeechRecognition`. `ttsService.js` uses `window.speechSynthesis`. `cleanTextForSpeech()` uses regex to strip markdown formatting, emojis, and links before reading replies.

Code Location: `client/src/components/chat/ChatInput.jsx:14` & `client/src/services/ttsService.js:45`

PART 21 — THE MOST LIKELY 15-MINUTE INTERVIEW SIMULATION

Time	Interviewer Question	Ideal Candidate Response	Trap / Mistake To Avoid
0:00 - 1:30	"Tell me about your project."	Use 60-second pitch: Full-stack AI appointment booking platform for salons using Groq LLaMA 3.3 70B, Node/Express, Supabase, and Google Calendar API.	Don't spend 5 minutes listing generic software engineering tools.
1:30 - 4:00	"Walk me through the architecture."	Explain separation: AI extracts intent -> Express enforces business rules -> Supabase stores state -> Google Calendar syncs external events.	Don't claim the LLM directly queries the database or decides slot availability.
4:00 - 6:30	"How does the AI parse user intent?"	Detail Groq SDK with LLaMA 3.3 70B, temperature 0.2, dynamic system prompt injection, JSON mode, and Zod schema validation with fallback models.	Don't forget to mention Zod schema validation and model fallback cascading.
6:30 - 9:00	"How do you check slot availability?"	Explain 30-min slot math (9 AM - 6 PM), dual-layer conflict checking (Supabase DB + GCal busy intervals), and 'anyone is fine' ranking via findBestAvailable().	Don't confuse slot availability with service capability (employee_services check).
9:00 - 11:30	"How does Google Calendar sync work?"	Explain OAuth 2.0 flow, offline token storage in tokens/google-tokens.json, busy interval checks, and event creation with 24h/30m client reminders.	Don't claim you used a Service Account if the code actually uses OAuth 2.0.
11:30 - 13:30	"What happens if two users book the same slot at once?"	Explain pre-check availability verification + atomic Supabase insert enforced by appointments_employee_slot_unique index (error code 23505).	Don't say 'it will never happen' — explain database unique constraints.
13:30 - 15:00	"What would you improve for production?"	Mention multi-tenant salon support, per-employee Google Calendars, Redis slot locking, JWT admin auth, and SMS notifications via Twilio.	Don't say 'the project is 100% production ready' without acknowledging trade-offs.

PART 23 — ACTUAL PROJECT LIMITATIONS & HONEST SHORTS

Things You Should NOT Pretend Are Implemented:

- Single Business Instance:** Currently hardcoded for 1 salon business ("SalonAI"). Multi-tenant salon organization IDs are not in DB schema.
- Single Primary Google Calendar:** Calendar sync connects to 1 primary Google Calendar ID (salon owner), rather than 3 separate calendar IDs for Sarah, Emma, and David.
- Local Token & Passphrase Storage:** Google OAuth tokens stored in local JSON file (tokens/google-tokens.json) and Admin passphrase checked via local session storage rather than JWT tokens.

4. **EmailJS Client Trigger:** Email confirmations dispatched from frontend browser SDK rather than backend queue worker (e.g. BullMQ / Redis).

How To Phrase This In An Interview: "For our demo MVP, we focused on perfecting intent extraction, dual-layer slot availability, and real-time calendar integration. For a enterprise multi-tenant rollout, we would migrate token storage to AWS Secrets Manager, add organization tenant IDs, and use Redis distributed locks for slot booking."

PART 25 — GUIDED 20-MINUTE CODE WALKTHROUGH PATH

Follow this exact path when opening the repository during an interview:

1. Open server/src/index.js -> Show Express setup, CORS, and route mounts.
2. Open server/src/ai/groq.js -> Point out MODEL_CASCADE array and extractIntent() function with Zod validation.
3. Open server/src/ai/prompts.js -> Show dynamic date calculation and prompt injection.
4. Open server/src/controllers/chatController.js -> Walk through handleChat() intent switch statement and confirmBooking().
5. Open server/src/services/availabilityService.js -> Highlight getAvailableSlotsForEmployee() and hasCalendarConflict().
6. Open server/src/services/calendarService.js -> Explain OAuth token storage, createCalendarEvent(), and reminders.
7. Open server/src/db/migration_salon.sql -> Show employee_services junction table and appointments_employee_slot_unique index.
8. Open client/src/pages/ChatPage.jsx & AdminPage.jsx -> Demonstrate frontend Bento UI, voice TTS, EmailJS trigger, and admin portal.